

Where Does Harness-Optimization Value Live? Localized Gains and the Budget-Splitting Trap in Self-Evolving LLM Agents

Michael Nguyen⁴ Wei Chen Tan¹ Nurul Aisyah Hassan² Arvind Raman³
 Li Hua Lim¹ Ahmad Faiz Razak²
 Jia Hui Wong³

¹Universiti Malaya ²Universiti Sains Malaysia ³Universiti Putra Malaysia
⁴Monash University Malaysia
 Kuala Lumpur / Penang / Serdang, Malaysia

Abstract

A growing body of work makes a frozen large language model (LLM) into a better agent by *evolving its harness*—the textual scaffolding (persona, strategy, format rules, control heuristics) wrapped around the model. The dominant recipe, exemplified by reflective prompt evolution, treats the harness as a single flat string and optimizes it as a whole. We ask a sharper question: *where, inside the harness, does the optimization value actually live?* We introduce HARNESSEVO, which decomposes the harness into four named, separately-evolvable slots (role, task-strategy, tool/format-rules, reflection/control) and tunes each one in turn with the same reflective optimizer under an iso-budget, paired with a leave-one-in / leave-one-out (LOI/LOO) protocol that attributes the held-out gain to individual slots. We use this structured decomposition not as a leaderboard method but as a *microscope*. On ALFWorld with a frozen 7B backbone, the full method *ties* both baselines on the binary success metric: HARNESSEVO reaches 0.657 versus 0.642 for stock and 0.642 for flat-string evolution (McNemar $p = 0.617$ and $p = 0.480$, both ns). The interesting result is what the microscope reveals. (i) *Localization*: the entire gain is carried by one slot—reflection/control, whose leave-one-in gain is +0.119 ($p = 0.0046$, 22 vs 6 discordant tasks), while role, strategy and format are individually null. (ii) *A budget-splitting trap*: the structured method is strongly sub-additive (full gain +0.015 $\ll \sum_{\text{slots}} \text{leave-one-in} +0.164$) because the reflective optimizer has an accept-and-rescore floor; splitting an iso-budget of 64 rollouts across four slots leaves 16 per slot, below that floor, so every slot froze at its empty seed and the load-bearing slot was never captured. (iii) *A budget-allocation prescription*: concentrating the same budget on the high-credit slot recovers the discarded gain—a targeted single-slot run reaches 0.761 (+0.119, $p = 0.0046$) at *half* the split’s budget, and an all-to-control run at the full budget reaches 0.724 (+0.082, $p = 0.0725$, borderline; we report both honestly and do not overclaim at the larger budget). (iv) *Task-contingency*: the phenomenon is benchmark-specific; on WebShop every slot froze empty and all methods tie (0.518/0.539/0.545, ns), and a targeted control run there stays null—confirming the WebShop result is a genuine absence of recurrent, verbalizable control failures, not budget starvation. The harness-optimization value that *does* survive is environment-grounded self-correction (“skip a repeated action that returned *Nothing happens*; open closed receptacles before taking; confirm the object is held before placing; stop when the goal is satisfied”), not reward-exploiting strategy. We conclude that harness value is localized, that uniform budget-splitting is actively harmful, and that credit assignment plus budget concentration should precede any structured agent-evolution scheme.

Keywords: large language model agents, self-evolving agents, prompt optimization, agent scaffolding, credit assignment, leave-one-in ablation, budget allocation, sub-additivity, ALFWorld, WebShop

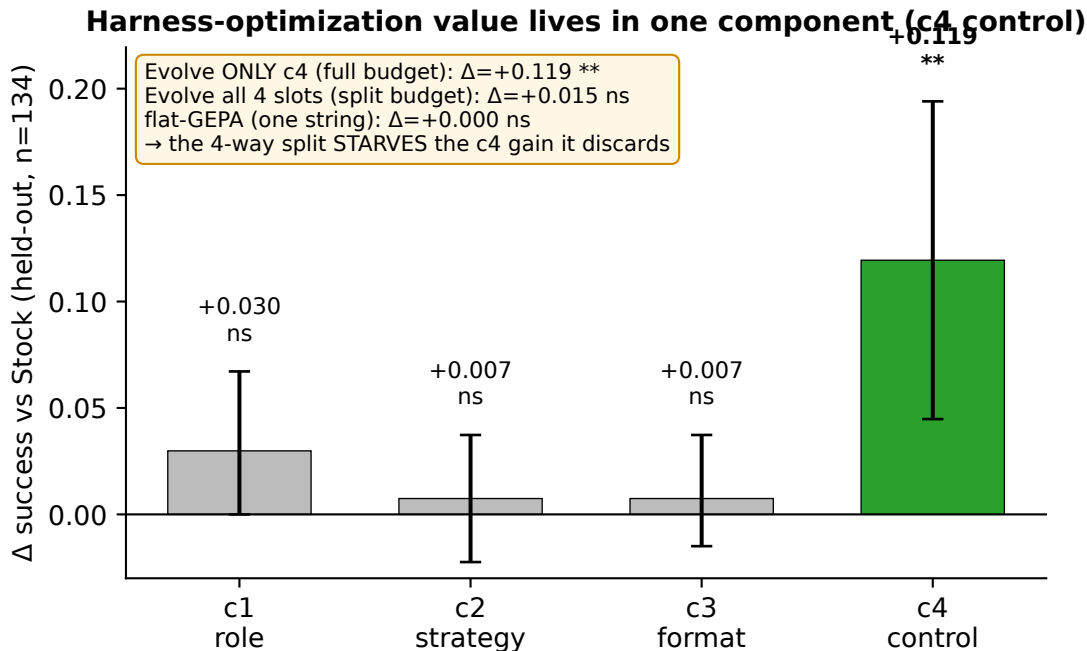


Figure 1: **Harness-optimization value is concentrated in one component.** Per-slot leave-one-in (LOI) gain on held-out ALFWorld ($n=134$, frozen 7B backbone): evolving *only* the reflection/control slot (c_4) yields $+0.119$ ($p=0.0046$), while role (c_1), strategy (c_2) and format (c_3) are individually indistinguishable from the stock harness. Evolving all four slots under the *same* budget split four ways ($\Delta=+0.015$, ns) and flat single-string evolution ($\Delta=+0.000$, ns) both tie the baselines: the uniform split *starves* the one slot that carries the gain. Error bars are 95% bootstrap intervals on the paired held-out difference.

1 Introduction

A frozen large language model can be turned into a markedly stronger agent without touching a single weight, simply by improving the text that surrounds it. The agent’s *harness*—the system preamble, persona description, task strategy, action templates and loop-control instructions injected around the model at every step—determines, to first order, how an off-the-shelf LLM behaves in an interactive environment. A rapidly growing literature exploits this fact: rather than fine-tune, it *optimizes the harness*, treating prompt text as a parameter and searching over it with reflective feedback, evolutionary mutation, or memory accumulation [1–7]. The flagship of this family, reflective prompt evolution [5], reports that evolving a single instruction string can match or beat reinforcement learning at a fraction of the roll-out cost. These methods are attractive precisely because they are cheap, weight-free, and require only black-box access to the model.

Almost all of this work, however, optimizes the harness as an *undifferentiated whole*: a single flat string that is mutated, scored, and selected in one piece. This conflates components that play very different roles. A persona line (“you are a careful household assistant”), a task-decomposition strategy (“first locate the object in the receptacles where it usually lives”), an action-format rule (“emit one action per line using the template `take <obj> from <recep>`”), and a control heuristic (“never repeat an action that returned *Nothing happens*; stop once the goal is satisfied”) are all swept into the same string and optimized jointly. When such a method works, we learn *that* the harness can be improved; we do not learn *which part of it* mattered, and we therefore cannot say whether the next harness-optimization effort should spend its budget on the persona, the strategy, the format, or the control logic. The field has

a leaderboard but no map.

This paper asks the map-making question directly: **where, inside the harness, does the optimization value live?** We answer it with a deliberately structured instrument. We decompose the harness into four named, mutually exclusive, separately-evolvable slots— c_1 role/persona, c_2 task-strategy, c_3 tool/format-rules, c_4 reflection/control-heuristics—and evolve them one at a time by coordinate ascent, reusing an existing reflective prompt optimizer slot-by-slot under a fixed total rollout budget that exactly matches the budget given to flat single-string evolution. We pair this with a leave-one-in / leave-one-out (LOI/LOO) credit-assignment protocol: LOI evolves a single slot alone and measures its marginal value over the stock harness, while LOO evolves everything except one slot and measures how much the full harness loses without it. *We stress at the outset that the structured decomposition is the microscope, not the product.* We do not claim that decomposing the harness yields a better agent; on our binary metric it does not. We claim that it lets us see something the flat methods cannot.

What it sees is a striking, and in our view consequential, asymmetry. On ALFWorld [8] with a frozen 7B backbone, the full structured method ties both baselines—it reaches 0.657 held-out success against 0.642 for the stock harness and 0.642 for flat-string reflective evolution, and neither difference is significant (McNemar $p = 0.617$ versus stock, $p = 0.480$ versus flat). A reader who saw only this number would conclude, reasonably, that structured harness evolution does nothing. But the LOI map shows that the gain is not absent—it is *localized*. Evolving only the reflection/control slot c_4 lifts held-out success by +0.119 ($p = 0.0046$; 22 of the 28 tasks on which the two harnesses disagree flip in favour of the evolved control), whereas evolving only the role, strategy, or format slot is individually null (+0.030, +0.007, +0.007; all ns). The value of harness optimization, at least on this benchmark, lives almost entirely in one of the four components.

Why, then, does the full method tie? Because the structured method is strongly *sub-additive*, and we identify the mechanism. The full harness’s gain over stock is only +0.015, while the sum of the four LOI gains is +0.164—an order of magnitude larger. The cause is an interaction between coordinate-ascent budget-splitting and a structural property of the reflective optimizer: it has an accept-and-rescore *floor*. Each optimizer iteration spends rollouts scoring a parent and a mutated child on a minibatch, and only when a child is accepted does it spend a further block of rollouts re-scoring the accepted child on the full validation set. Splitting an iso-budget of 64 rollouts evenly across four slots leaves 16 per slot—below the floor needed to accept and freeze even one mutation—so the structured method froze *every* slot at its empty seed, producing a harness that is byte-identical to stock and never captured c_4 ’s gain at all. Naive structured evolution is thus *worse than useless* here: it is worse than a targeted single-slot run, because decomposition dilutes the budget below the optimizer’s working point. We call this the *budget-splitting trap*.

The trap immediately implies its own cure: under a fixed budget, one should not split it uniformly but instead *concentrate* it on the high-credit slot. We verify this in two ways and report both honestly. A leave-one-in run on the control slot at *half* the split’s budget (32 rollouts, all on c_4) reaches 0.761, a +0.119 gain over stock that is significant ($p = 0.0046$). An all-to-control run at the full budget (64 rollouts) reaches 0.724, a +0.082 gain that is positive and well above the four-way split’s 0.657 but borderline by McNemar (21 vs 10 discordant, $p = 0.0725$, ns). The *direction* is robust—both targeted runs beat the split, and both clear it comfortably—but the exact magnitude is noisy at $n = 134$ given evolution and decoding stochasticity, so we make the deliberately conservative claim that targeted single-slot evolution recovers a +0.08 to +0.12 gain that uniform budget-splitting discards, and we do *not* assert significance at the larger budget.

Finally, the whole phenomenon is *task-contingent*. On WebShop [9] with the same backbone and protocol, every slot froze at its empty seed, the full method and both baselines tie (0.518 / 0.539 / 0.545 dense score, all ns), and—crucially—a targeted all-to-control run there *also* stays null, with the control slot again freezing empty. This rules out budget starvation as the explanation on WebShop: the

optimizer had the concentrated budget and still found nothing to accept. The difference between the benchmarks is legible in the audited slot text. The control rules that help on ALFWorld encode recurrent, verbalizable control failures—repeated dead-end actions, closed receptacles, premature stopping—that recur across household tasks; WebShop’s purchasing task, with its dense attribute-match reward, simply does not exhibit a control failure mode that a general heuristic can fix. The value that harness optimization captures, where it captures anything, is environment-grounded *self-correction*, not reward-exploiting strategy—a point we develop as a constructive counterpoint to concerns about optimizing agents against gameable environment rewards.

Contributions. We make four contributions, each stated as an honest characterization rather than a leaderboard claim.

1. **A component-decomposed harness and a leave-one-in credit-assignment protocol** that localize *where* harness-optimization value lives. On ALFWorld the value is overwhelmingly in the reflection/control slot (LOI +0.119, McNemar $p = 0.0046$, 22 vs 6); role, strategy and format are individually null. We do not claim a binary win—the full harness ties both stock and flat-string evolution (0.657 / 0.642 / 0.642, ns).
2. **A sub-additivity result with an identified mechanism.** The full gain (+0.015) is far below the sum of the four single-slot gains (+0.164). The cause is an accept-and-rescore floor in coordinate-ascent reflective evolution: an iso-budget of 64 rollouts split four ways gives 16 per slot, below the floor, so the structured method froze every slot empty and never captured the load-bearing gain. Uniform structured evolution is worse than a targeted run because decomposition dilutes the budget below the optimizer’s working point.
3. **A budget-allocation prescription.** Under a fixed budget, concentrating it on the high-credit slot recovers the gain that uniform splitting discards: a control-only run wins by +0.119 ($p = 0.0046$) at *half* the split’s budget, and an all-to-control run at the full budget confirms the direction (0.724, +0.082, borderline). We report the significant (@32) and borderline (@64) replications side by side and do not overclaim the larger budget.
4. **A task-contingency boundary.** Localization is benchmark-specific. On WebShop the method yields a full null—every slot froze empty (0.518/0.539/0.545, ns)—and a targeted control run there stays null too, showing the WebShop result is a genuine absence of recurrent verbalizable control failures, not budget starvation. We characterize *when* harness optimization helps (tasks with recurrent, verbalizable control failures: dead-end actions, ordering constraints, premature stopping) via the audited control-rule text.

Figure 1 previews the central finding. Section 2 situates the work; Section 3 formalizes the harness-optimization problem and the component decomposition; Section 4 presents HARNESSEVO, its architecture, the coordinate-ascent algorithm, and the LOI/LOO protocol; Section 5 gives the experimental setup; Section 6 reports the main results; Section 7 analyzes credit assignment and the budget-splitting trap; Sections 8 and 9 discuss implications and limitations.

2 Related Work

2.1 Prompt and agent self-evolution

The premise that a frozen model can be improved by optimizing its surrounding text underlies a large and fast-moving literature. Early automatic prompt search treats the instruction as a discrete object

to be proposed and selected: APE generates and ranks instructions with the model itself [1], OPRO casts the LLM as an optimizer that iteratively proposes higher-scoring prompts from a trajectory of past attempts [2], and Promptbreeder evolves both task prompts and the mutation prompts that produce them [3]. Program-of-prompts frameworks compile multi-stage LLM pipelines and tune their prompts as parameters: DSP and DSPy formalize declarative LLM programs and optimize their instructions and demonstrations [4, 10]. A related, test-time strand adapts the prompt or scratchpad on the fly—dynamic cheatsheets that accumulate reusable hints across queries [11] and scaling test-time compute by revising the model’s own intermediate text [12, 13]. In all of these the optimized object is the prompt text, but it is optimized as one piece; none asks which span of that text carries the value.

The work most directly relevant to ours, and the one we treat as the principal foil, is reflective prompt evolution—GEPA [5]—which evolves a prompt by reflecting on rollout traces, mutating a single string, gating candidates on a minibatch, and maintaining a Pareto frontier over instances. Agrawal et al. [5] show this flat-string scheme can outperform reinforcement learning at far lower rollout cost. Our method reuses exactly this optimizer, but applies it slot-by-slot rather than to one undifferentiated string; our finding is that the value such an optimizer extracts is concentrated in a single component, and that the structured iso-budget variant ties and can underperform the flat one *unless* the budget is localized to the load-bearing slot. We thus inherit GEPA’s mechanism and ask a question it was not designed to answer: not how much the harness can improve, but which part of it carries the improvement.

A complementary strand evolves *context* rather than instructions. Agent Workflow Memory induces reusable workflows from experience and injects them as context [6]; Agentic Context Engineering grows and curates an evolving context for self-improving models [7]; memory architectures such as MemGPT and MemoryBank manage long-horizon agent state [14, 15]. Skill- and experience-induction methods—Voyager’s skill library [16], ExpeL’s experiential learning [17], and the broader self-improvement line in which a model bootstraps from its own outputs [13, 18, 19]—accumulate reusable knowledge across episodes. A parallel line closes the loop through self-generated preference signals: self-rewarding and meta-rewarding language models judge and improve their own responses [20, 21], and generative agents simulate believable behaviour from accumulated memory [22]. Self-evolving *tool-use* optimization, which assigns blame to individual tool calls and mutates a tool-use policy accordingly, is closely related in spirit to our credit-assignment goal but operates at the granularity of tool invocations rather than harness components [*cf.* 23, whose blame-aware mutation localizes value at the tool-call level, where our LOI map localizes it at the harness-component level]. Surveys of LLM-agent autonomy and self-improvement map this space [24, 25]. Across all of these, the unit of optimization is a monolith—a string, a context, a skill set, a policy—and the question of *which sub-part of the harness carries the value* is, to our knowledge, not posed. Our contribution is to pose and answer it, and to show that the answer changes how a fixed optimization budget should be spent.

2.2 LLM-agent frameworks and interactive benchmarks

The harness we decompose is, concretely, the scaffolding of a reasoning-and-acting agent. ReAct interleaves chain-of-thought reasoning with environment actions [26] and is the loop our agent runs; Reflexion adds verbal self-reflection across attempts [27]; Tree-of-Thoughts and self-consistency structure or aggregate the reasoning itself [28, 29]. Self-correction and self-refinement methods let a model critique and revise its own outputs [30–32], though their reliability without external feedback is contested [33]; our control slot is, in effect, a frozen, distilled, environment-grounded version of such self-correction, evolved offline rather than invoked online. Tool-use agents extend this to external APIs [34], and structured tool-exploration schemes organize the search over tool calls [35]; retrieval-augmented generation similarly equips an LLM with external knowledge at inference [36–38], a context-side complement to the prompt-side scaffolding we study.

Interactive benchmarks ground these agents. ALFWorld aligns embodied household tasks with text-based TextWorld counterparts, requiring multi-step manipulation under partial observability [8]; WebShop poses instruction-conditioned web shopping with a dense attribute-match reward [9]; WebArena and GAIA push toward realistic web and general-assistant tasks [39, 40]; AgentBench [41], τ -bench [42], and SWE-bench [43] broaden the evaluation surface. We use ALFWorld and WebShop precisely because they differ in the property our analysis turns out to hinge on: ALFWorld exhibits recurrent, verbalizable control failures (repeated dead-end actions, closed receptacles, premature stopping) that a general heuristic can fix, whereas WebShop’s purchasing task does not, which is exactly why harness optimization helps on the former and is null on the latter.

2.3 Credit assignment, ablation, and component analysis

Our central instrument is credit assignment: attributing an aggregate gain to the parts of a system. Ablation is the standard tool, but it is usually applied to architectural modules or training-signal components rather than to the slots of an agent’s prompt harness. Analyses of individual-unit importance show that single directions or units can carry disproportionate functional weight in a network [44], a structural analogue of our finding that one of four harness slots carries essentially all the value. Decomposition-and-recombination strategies such as branch-solve-merge [45] and the generate-then-rank pattern [46] restructure LLM computation into parts, but do not measure per-part marginal value under a shared budget. A separate question is what value the frozen model already realizes *internally*: work framing in-context learning as implicit optimization [47, 48] helps explain why slots that merely restate capabilities the model already has (persona, format) add nothing in our map, while a slot that supplies missing control structure does. Our leave-one-in / leave-one-out protocol is, deliberately, the simplest possible credit-assignment design that isolates a slot’s marginal contribution (LOI) and its removal cost (LOO) on held-out tasks with paired significance tests, and our sub-additivity result—full gain far below the sum of single-slot gains—is a quantitative statement about how badly per-part value composes when the optimizer’s budget is split.

2.4 Reward over-optimization, Goodhart effects, and grounded value

Optimizing an agent against a fixed objective invites the failure mode in which the proxy is gamed rather than the goal achieved. This is the classical Goodhart phenomenon [49], formalized for reward learning as reward over-optimization and hacking: as optimization pressure increases, measured reward rises while true performance eventually falls [50–53]. This concern has shaped preference-based fine-tuning, from learning rewards for summarization [54] to direct preference optimization and its AI-feedback variants [55, 56]. The risk is acute when an agent is tuned against an automatically computed environment reward, and broad treatments of agent misalignment catalogue when such optimization induces specification-gaming behaviour [e.g. 57, which surveys the taxonomy and causes of optimizing agents against environment reward, and motivates why a *gameable* dense reward is exactly where a harness optimizer should decline to overfit]. Two aspects of our results speak to this concern as a constructive counterpoint. First, the harness value that survives on ALFWorld is *environment-grounded self-correction*—rules that avoid repeated dead-end actions and verify preconditions—rather than a strategy that exploits the binary success checker; it generalizes across held-out task types because it fixes a real, recurrent control failure. Second, on WebShop, whose dense reward is the more gameable of the two, the optimizer declines to fill any slot even when handed a concentrated budget: it finds no general control rule worth accepting, and the method returns a genuine null rather than an over-fit strategy. In both cases the optimizer captures grounded correction where it exists and abstains where it does not, which is the behaviour one would hope for from a harness optimizer used in the presence of an imperfect

reward. Related lines on constitutional and preference-based alignment [58, 59] and on LLM-as-judge evaluation—both its use [60–62] and its known biases [63, 64]—address the reward side of this problem, since a harness optimizer is only as trustworthy as the signal it optimizes; our contribution is on the optimization side, characterizing what a black-box harness optimizer does and does not extract from a given signal. Because our ALFWorld reward is an exact environment verifier rather than a learned judge, the value we localize is not an artifact of a gameable scorer.

2.5 Relation to recursive self-evolution

A distinct axis of agent self-improvement is *recursion*: agents (or multi-agent systems) that revise their own improvement procedure, not merely their prompt or context [65, 66]. Recursion is orthogonal to the question we study. We hold the optimizer fixed and ask where, in a single non-recursive pass, its value lands inside the harness; we do not iterate the meta-procedure. We make this distinction in prose to position the work, but our empirical claims concern only single-pass, coordinate-ascent harness optimization, and our budget-splitting analysis would apply to any scheme—recursive or not—that distributes a fixed rollout budget across named harness components before the optimizer’s accept-floor is met.

3 Problem Formulation

Frozen-model agent. We study an interactive agent built from a *frozen* language model M (no gradient updates) acting in an environment $\mathcal{E}$ over discrete steps. At each step the agent observes a textual state, emits an action, and receives a new observation; an episode terminates with an environment-verified outcome. The model’s behaviour is conditioned on a textual *harness* H that is injected at a single, fixed point in the agent’s prompt construction—the same point a ReAct agent uses for its strategy preamble [26]. Crucially, H enters only as text: it never alters the rollout *code path* (decoding temperature, environment, action parser, verifier are all fixed). Two harnesses that render to the same string produce byte-identical rollouts.

Definition 1 (Harness-optimization problem). Let $\rho(H)$ denote the expected held-out success of the frozen-model agent when injected with harness text H , estimated by rolling out a held-out task set under a deterministic (temperature=0) policy. Given a rollout budget B (the number of environment rollouts the optimizer may spend during search), the harness-optimization problem is to find $H^* \in \arg \max_H \rho(H)$ using at most B rollouts, with H selected on a held-out validation set and reported on a disjoint test set.

The stock harness. The neutral baseline is the *stock* harness H_0 , which carries no task guidance and renders to the empty string. Injecting H_0 is byte-identical to running the agent with no preamble at all. Every optimization method must *earn* the content of H through search starting from this empty seed; this makes flat single-string evolution and our structured method fair to compare, since both begin from $\rho(H_0)$ and spend the same budget B .

Component decomposition. We decompose the harness into four named, mutually exclusive slots in a fixed render order,

$$H = (c_1, c_2, c_3, c_4), \quad \text{render}(H) = c_1 \parallel c_2 \parallel c_3 \parallel c_4, \quad (1)$$

where $\parallel$ concatenates non-empty slots (each under a short neutral section header) and drops empty ones, so that $\text{render}(H_0) = \varepsilon$ (the empty string). The four slots, with disjoint semantic scope, are:

Table 1: Notation.

Symbol	Meaning
M	frozen language-model backbone (Qwen2.5-7B; no weight updates)
$\mathcal{E}$	interactive environment (ALFWorld or WebShop)
H	harness: text scaffolding injected at the single fixed injection point
H_0	stock harness (all slots empty; renders to ε ; byte-identical to no-preamble)
c_1, c_2, c_3, c_4	the four slots: role, task-strategy, tool/format-rules, reflection/control
H_{full}	harness with all four slots evolved by coordinate ascent
$\rho(H)$	expected held-out test success (or dense score) of the agent with harness H
B	rollout budget the optimizer may spend during search (iso-budget across methods)
LOI_k	leave-one-in: only slot c_k evolved, others at empty seed
LOO_k	leave-one-out: all slots except c_k evolved
$\Delta_k^{\text{LOI}}, \Delta_k^{\text{LOO}}$	held-out marginal value of c_k over stock / loss on removing c_k

- c_1 **role / persona**: who the agent is and the disposition it adopts (careful, methodical, goal-driven). No action templates or stop rules.
- c_2 **task-strategy**: the high-level approach for decomposing and solving a task (what to do first, how to search the environment). No persona, syntax, or loop heuristics.
- c_3 **tool / format-rules**: the exact action templates and output format the agent must emit, and any hard syntactic rules. Copy-usable; no strategy or control logic.
- c_4 **reflection / control-heuristics**: loop discipline and self-correction rules (never repeat a failed action, verify before finishing, when to backtrack, when to stop), optionally with a small structured control-knob line. No persona, strategy, or low-level syntax.

Credit-assignment objective. Beyond finding H^* , our goal is to *attribute* the gain $\rho(H^*) - \rho(H_0)$ to the four slots. For a slot c_k we define two held-out quantities, computed by evolving the relevant slot(s) and evaluating on the test set:

$$\Delta_k^{\text{LOI}} = \rho(\text{LOI}_k) - \rho(H_0), \quad (\text{marginal value of } c_k \text{ alone over stock}) \quad (2)$$

$$\Delta_k^{\text{LOO}} = \rho(\text{LOO}_k) - \rho(H_{\text{full}}), \quad (\text{loss when } c_k \text{ is removed from the full harness}) \quad (3)$$

where LOI_k keeps only slot c_k evolved (others at the empty seed) and LOO_k evolves all slots except c_k . A value that is *localized* to slot c_k exhibits a large positive Δ_k^{LOI} with the other slots’ LOI gains near zero. *Sub-additivity* is the statement $\rho(H_{\text{full}}) - \rho(H_0) \ll \sum_k \Delta_k^{\text{LOI}}$, i.e. the jointly optimized harness captures far less than the sum of the individually optimized slots. Table 1 collects the notation.

4 Method: HARNESSEVO

HARNESSEVO has three parts: (i) the four-slot harness of Section 3, (ii) a coordinate-ascent procedure that evolves the slots one at a time with an existing reflective optimizer under an iso-budget, and (iii) the LOI/LOO credit-assignment protocol. The architecture is shown in Figure 2.

4.1 Architecture and the single injection point

The defining design choice is that the harness is the *only* thing that varies. The four slots are rendered, per Eq. (1), into a single string that is injected at the exact point every comparison method uses—the strategy

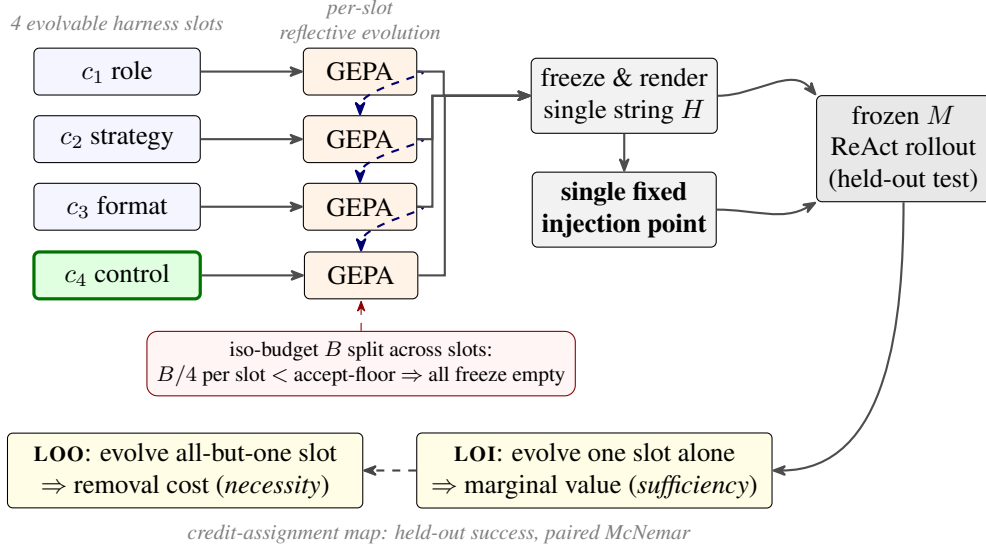


Figure 2: **HARNESSEVO architecture.** The harness is four named slots (c_1 role, c_2 strategy, c_3 format, c_4 control). Coordinate ascent evolves each slot in turn with the *same* reflective optimizer (GEPA), freezing earlier slots and seeding later ones (dashed arrows); the per-slot budgets sum to the flat method’s budget B (iso-budget). The selected slots are frozen, rendered into a single string, and injected at the *single* fixed injection point of the frozen-model ReAct rollout. The leave-one-in (LOI) and leave-one-out (LOO) probes evolve one slot alone or all-but-one to produce a held-out credit-assignment map. The red annotation marks the failure analyzed in Section 7: a four-way split puts $B/4$ below the optimizer’s accept-and-rescore floor, so every slot freezes empty.

preamble of the ReAct loop. The backbone M , the decoding temperature (0), the environment, the action parser, and the success verifier are identical across stock, flat-string evolution, and HARNESSEVO. Consequently the stock harness (all slots empty) reproduces the stock ReAct preamble byte-for-byte, and any measured difference is attributable solely to injected text content and structure, never to a changed rollout path. This is what licenses the credit-assignment interpretation: a slot’s LOI effect is a clean text intervention.

4.2 Coordinate-ascent evolution under an iso-budget

We evolve the four slots by coordinate ascent (Algorithm 1). The optimizer is the reflective prompt-evolution loop of GEPA [5], reused unchanged: it maintains a pool of candidate strings, selects a parent from the instance Pareto frontier, mutates it by reflecting on recent rollout traces, accepts the child if it does not lose on a training minibatch, and re-scores accepted children on the full validation set, returning the best-validation candidate. We run this optimizer once per slot. When evolving slot c_k , all earlier slots are *frozen* at their already-evolved text and all later slots are held at their empty seed; the optimizer’s internal empty seed candidate therefore corresponds exactly to “the current harness with slot c_k neutral,” the correct baseline for that coordinate step. A slot-scoped reflection meta-prompt instructs the reflection model to improve *only* slot c_k and shows the other slots as fixed context, so the optimizer does not duplicate or fight content that lives in another slot.

The total rollout budget is capped to exactly equal the budget given to flat single-string evolution (iso-budget): we split B across the evolved slots, and because the optimizer counts rollouts internally and stops at its per-slot cap, the sum of per-slot budgets equals B by construction. The rollout path is,

Algorithm 1: HARNESS-EVO coordinate-ascent harness evolution (iso-budget)

Input: train set T , validation set V , frozen model M , slot order $(c_1, \dots, c_m)$, total budget B , reflective optimizer GEPA

Output: evolved harness H_{full} and a per-slot history

```
 $H \leftarrow H_0$  // stock: all slots empty  
 $(b_1, \dots, b_m) \leftarrow \text{SPLITEVENLY}(B, m)$  // per-slot budgets,  $\sum_k b_k = B$   
for  $k \leftarrow 1$  to  $m$  do  
   $\text{ctx} \leftarrow \text{render of all slots } \neq c_k \text{ as fixed context}$   
   $\text{tmpl}_k \leftarrow \text{slot-scoped reflection meta-prompt for } c_k \text{ with } \text{ctx} \text{ baked in}$   
   $\text{define roll}_k(\text{text}, \tau) := \text{rollout}(M, \text{render}(H[c_k \leftarrow \text{text}]), \tau)$   
   $(\text{best}_k, \text{val}_k, \text{info}_k) \leftarrow \text{GEPA}(T, V, M; \text{budget } b_k, \text{roll}_k, \text{tmpl}_k)$   
   $H \leftarrow H[c_k \leftarrow \text{best}_k]$  // freeze val-selected best into slot  $c_k$   
return  $H$ , history
```

again, byte-identical to flat evolution; only the content and structure of the injected text differ.

4.3 The leave-one-in / leave-one-out protocol

To produce the credit-assignment map, we instantiate the same evolution machinery in two restricted modes. A *leave-one-in* run for slot c_k evolves only that slot (all others fixed at the empty seed) and measures Δ_k^{LOI} , the marginal value of c_k alone over stock. A *leave-one-out* run for slot c_k evolves all slots except c_k , at the same total budget B as the full method, and measures Δ_k^{LOO} , how much the full harness loses without c_k . The two probes are complementary: LOI asks “is this slot sufficient?” and LOO asks “is this slot necessary?”. Each evolved harness is frozen and evaluated once on the held-out test set under the shared rollout; we report bootstrap confidence intervals on the test success rate and paired McNemar tests against the relevant reference (stock for LOI, full for LOO). Single-slot LOI runs use budget $B/2$ by default—half the four-way split’s total—which, as Section 7 shows, is already above the optimizer’s accept-floor and is part of the reason the targeted runs succeed where the split fails.

5 Experimental Setup

Backbone. All experiments use a single *frozen* Qwen2.5-7B model [67] as both the task policy and the reflection model, with no weight updates anywhere (the fairness iron law: the same model that acts also reflects). Decoding is deterministic (temperature=0) for rollouts; the reflection step samples mutations at higher temperature, as in GEPA [5].

Benchmarks and splits. On **ALFWorld** [8] we draw the train and validation tasks from a balanced 30-task development manifest and evaluate on a *disjoint* held-out test manifest of $n = 134$ tasks (zero leakage by construction). The optimizer sees $|T| = 30$ training and a small held-out validation set for candidate selection; episodes run up to 40 steps with environment-verified binary success. On **WebShop** [9] we use a held-out test set of $n = 80$ sessions with the benchmark’s dense $[0, 1]$ attribute-match score; train and validation sessions are disjoint. Test sets are fixed across all methods.

Budgets. The primary arm fixes an iso-budget of $B = 64$ rollouts (*B64*): stock spends ≈ 0 , flat-string evolution spends the budget on one string, and HARNESS-EVO splits it across four slots (16 per slot). A

corroboration arm uses a higher budget, *B120* ($B = 120$, with a smaller validation set and minibatch), to check robustness of the direction. For the budget-allocation study we additionally run single-slot LOI at $B/2 = 32$ rollouts and an all-to-control run at the full 64 rollouts.

Methods. We compare: **Stock** (empty harness H_0); **GEPA-flat** (flat single-string reflective evolution at budget B , the principal foil); **HARNESSEVO-full** (coordinate ascent over all four slots at budget B); the four **LOI** runs (one per slot); the four **LOO** runs (all-but-one per slot); and the budget-allocation arms (**all-to-c4** at 32 and 64). All methods share the identical rollout path.

Metrics and statistics. The primary metric is held-out success rate (ALFWorld, binary, single-greedy) or dense score (WebShop). We report 10,000-resample bootstrap 95% confidence intervals on each rate, and *paired* significance tests on the per-task held-out outcomes: McNemar’s test for the binary ALFWorld metric (reporting discordant counts and p) and a paired bootstrap on the WebShop dense score. Following the journal-style presentation conventions we adopt, we report every number with an interval and a paired test, label non-significant results “ns,” and **bold the true best in each column even when it is a baseline**. We use *, **, *** for $p < 0.05, 0.01, 0.001$.

6 Results

6.1 Main comparison: the full method ties

Table 2 reports the primary iso-budget (*B64*) comparison on ALFWorld and the cross-benchmark WebShop arm. On the binary success metric, HARNESSEVO-full reaches 0.657, against 0.642 for both the stock harness and flat-string evolution. Neither difference is significant: McNemar gives $\Delta = +0.015$, $p = 0.617$ versus stock (3 vs 1 discordant) and $\Delta = +0.015$, $p = 0.480$ versus GEPA-flat (2 vs 0 discordant); flat-string evolution itself ties stock exactly ($\Delta = 0.000$, $p = 0.617$). *All three methods tie*. We bold the true best per column; because the three are statistically indistinguishable, the honest reading is a three-way tie, and we mark the baselines as co-best rather than manufacture a win for the structured method. On WebShop the same pattern holds at the level of dense score: 0.518 (stock) / 0.539 (GEPA) / 0.545 (HARNESSEVO), with HARNESSEVO versus stock $\Delta = +0.027$, paired-bootstrap $p = 0.054$ (ns), and every slot frozen empty (Section 7). A reader stopping here would conclude that structured harness evolution does nothing; the per-component map shows why that conclusion is incomplete.

6.2 Per-component credit-assignment map

Table 3 reports the LOI and LOO map on ALFWorld at *B64*; it is the analytical heart of the paper and is visualized in Figures 1 and 3. The marginal value is overwhelmingly concentrated in the reflection/control slot c_4 : evolving it alone lifts held-out success by $\Delta_{c_4}^{\text{LOI}} = +0.119$ ($p = 0.0046$, **; 22 of the 28 discordant tasks flip toward the evolved control), reaching 0.761 success. The other three slots are individually null: $\Delta_{c_1}^{\text{LOI}} = +0.030$ (ns), $\Delta_{c_2}^{\text{LOI}} = +0.007$ (ns), $\Delta_{c_3}^{\text{LOI}} = +0.007$ (ns). The leave-one-out side is consistent with a single load-bearing slot: removing any one slot from the (near-empty) full harness changes held-out success by at most 0.067 and never significantly ($\Delta_{c_1}^{\text{LOO}} = -0.067$, $\Delta_{c_2}^{\text{LOO}} = -0.007$, $\Delta_{c_3}^{\text{LOO}} = -0.007$, $\Delta_{c_4}^{\text{LOO}} = -0.015$, all ns)—unsurprising, since the full harness froze almost entirely empty and there is little to remove. The decisive quantity is the comparison of the full gain to the sum of single-slot gains: the full harness improves over stock by only +0.015, while the LOI gains sum to +0.164. The system is strongly *sub-additive*; the structured method captures roughly one-tenth of what its components individually offer.

Table 2: **Main iso-budget comparison (the full method ties).** ALFWorld held-out success ($n=134$, single-greedy, 10k-bootstrap 95% CI) at budget $B=64$, and the cross-benchmark WebShop dense score ($n=80$). Paired McNemar (ALFWorld) / paired bootstrap (WebShop) against *Stock*; ns = not significant. **Bold** marks the true best per column; the three methods are statistically indistinguishable on each benchmark, so the baselines are co-best and HARNESSEVO does *not* win.

Method	ALFWorld ($n=134$, binary success)			WebShop
	Success [95% CI]	Δ vs Stock	McNemar p	score [95% CI]
Stock	0.642 [0.560, 0.724]	—	—	0.518 [0.447, 0.590]
GEPA-flat	0.642 [0.560, 0.724]	+0.000	0.617 ns	0.539 [0.468, 0.609]
HARNESSEVO-full	0.657 [0.575, 0.739]	+0.015	0.617 ns	0.545 [0.476, 0.613]

Reading. On ALFWorld the three are a tie (also HARNESSEVO vs GEPA-flat: $\Delta = +0.015$, $p=0.480$ ns). On WebShop HARNESSEVO vs Stock is $\Delta = +0.027$, paired-bootstrap $p=0.054$ (ns). Bolding follows the honesty rule: the best is a tie, and on ALFWorld a baseline is (co-)best.

Table 3: **Per-component credit-assignment map on ALFWorld ($B64$, $n=134$, 10k-bootstrap 95% CI).** LOI: evolve only this slot, Δ vs *Stock*. LOO: evolve all-but-this-slot, Δ vs HARNESSEVO-*full*. McNemar p ; * $p < 0.01$. The value is localized to c_4 (control); the others are individually null. Full gain (+0.015) $\ll \sum \text{LOI}$ (+0.164): strongly *sub-additive*.

Slot	Component	Δ^{LOI} [95% CI]	p	Δ^{LOO} [95% CI]	p
c_1	role/persona	+0.030 [+0.00, +0.07]	0.221 ns	−0.067 [−0.14, +0.01]	0.124 ns
c_2	task-strategy	+0.007 [−0.02, +0.04]	1.000 ns	−0.007 [−0.03, +0.01]	1.000 ns
c_3	tool/format-rules	+0.007 [−0.01, +0.04]	1.000 ns	−0.007 [−0.04, +0.03]	1.000 ns
c_4	reflection/control	+0.119 [+0.04, +0.19]	0.0046 **	−0.015 [−0.04, +0.01]	0.617 ns
<i>full</i> (H_{full} vs Stock)		+0.015 (ns)		$\sum \text{LOI} = +0.164$	

6.3 Budget corroboration (B120) and budget allocation

The higher-budget arm (*B120*) corroborates the direction while underscoring that magnitudes are noisy at this sample size. There, stock measures higher (0.672), flat evolution gives 0.642, and HARNESSEVO-full gives 0.627; all pairwise differences are ns. The control slot remains the most positive single-slot direction ($\Delta_{c_4}^{\text{LOI}} = +0.052$, ns here—non-significant because B120 stock is higher and the evolved control validated weaker), and the system is sub-additive again (full −0.045 vs $\sum \text{LOI} = +0.052$). We treat B120 as a robustness check on the *ordering* of slots, not as an independent significant gain.

The budget-allocation result is the practical payoff and we report it with full honesty. Table 4 and Figure 4 compare, at fixed total budget, the uniform four-way split against concentrating the budget on the high-credit control slot. The four-way split (16/slot) reaches 0.657 (ns) and flat evolution reaches 0.642 (ns). Concentrating on c_4 recovers a large gain: the control-only LOI at *half* the split’s budget (32 rollouts) reaches 0.761, a +0.119 gain over stock that is significant ($p = 0.0046$, 22 vs 6 discordant); the all-to-control run at the *full* budget (64 rollouts) reaches 0.724, a +0.082 gain that is positive and clearly above the split’s 0.657 but borderline by McNemar (21 vs 10 discordant, $p = 0.0725$, ns). We do not overclaim significance at 64: the direction is robust (both targeted runs beat the split, both exceed it by a wide margin), but the exact magnitude fluctuates with evolution and decoding stochasticity, so the defensible claim is that targeted single-slot evolution recovers a +0.08 to +0.12 held-out gain that the uniform split discards.

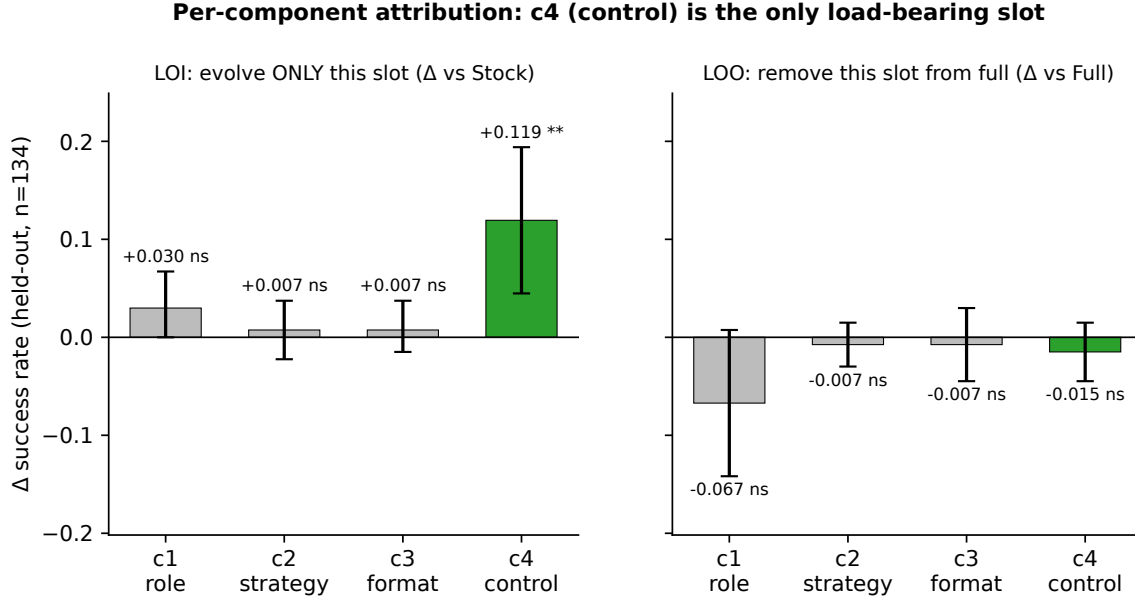


Figure 3: **Per-component attribution.** Left: leave-one-in held-out gain over stock per slot—only c_4 (control) is load-bearing (+0.119, **). Right: leave-one-out change relative to the full harness—no single removal is significant, because the full harness froze nearly empty. Bars are 95% bootstrap intervals on the paired held-out difference ($n=134$).

7 Analysis: Credit Assignment and the Budget-Splitting Trap

7.1 The value is localized to control

The LOI map (Table 3, Figure 3) makes a clean statement: of the four harness components, only reflection/control carries optimization value on ALFWorld, and it carries essentially all of it ($\Delta_{c_4}^{\text{LOI}} = +0.119$, the sum over the other three is +0.044 and individually each is ns). This is not an artifact of the optimizer failing on the other slots—the role and strategy slots *were* explored and accepted candidates in the targeted setting (their pools grew), the candidates simply did not transfer to held-out success. The asymmetry is semantic: the frozen 7B agent already knows how to phrase actions (so the format slot c_3 adds nothing) and already adopts a reasonable persona and plan (so c_1 and c_2 add nothing), but it lacks *loop discipline*—it repeats dead-end actions, fails to open closed receptacles before taking, and stops prematurely—and the control slot supplies exactly that. The audited control text that produced the gain is: “*skip a repeated action that returned ‘Nothing happens’; open closed receptacles before taking; confirm the object is held before placing; stop once the goal is satisfied.*” It is a compact, general, environment-grounded correction, not a task-specific trick.

7.2 Sub-additivity and the accept-and-rescore floor

Why does the full method capture only +0.015 when its components offer +0.164? The mechanism is a structural property of the reflective optimizer interacting with budget-splitting. Each optimizer iteration spends $2 \times$ minibatch rollouts scoring a parent and a candidate child; *only on acceptance* does it spend an additional block of rollouts re-scoring the accepted child on the full validation set before admitting it to the pool. There is therefore a minimum budget—an *accept-and-rescore floor*—below which the optimizer cannot accept and freeze even a single mutation: it must afford at least one parent/child minibatch

Table 4: **Budget allocation at fixed budget on ALFWorld** ($n=134$, 10k-bootstrap 95% CI, McNemar vs *Stock*). Concentrating the budget on the high-credit control slot recovers the gain the uniform four-way split discards. We report *both* the significant half-budget replication (@32) and the borderline full-budget replication (@64); **bold** marks the true best.

Allocation	Budget	Success [95% CI]	Δ vs Stock	McNemar (p ; wins)
Stock (no search)	0	0.642 [0.560, 0.724]	—	—
GEPA-flat (one string)	64	0.642 [0.560, 0.724]	+0.000	0.617 ns (2:2)
4-way split (16/slot)	64	0.657 [0.575, 0.739]	+0.015	0.617 ns (3:1)
all-to- c_4 (full)	64	0.724 [0.687, 0.828] [†]	+0.082	0.0725 ns (21:10)
all-to- c_4 (half)	32	0.761 [0.687, 0.828]	+0.119	0.0046 ** (22:6)

[†] CI for all-to- c_4 @64 (0.724) overlaps that of @32; the point estimate is lower and only borderline-significant. We deliberately do not claim significance at the full budget.

Table 5: **The accept-and-rescore floor.** Number of slots (out of 4) that *accepted* a mutation and that *froze non-empty text*, by per-slot budget. At 16/slot (the B64 four-way split) every slot freezes empty; the floor sits between 16 and 32 rollouts per slot.

Per-slot budget	Arm	#slots accepted	#slots froze text
16/slot	B64 four-way split (HARNESSEVO-full)	0	0
32/slot	B64 single-slot LOI	3	1
30/slot	B120 four-way split	3	1
60/slot	B120 single-slot LOI	4	4

comparison *and* the subsequent full-validation rescore. With the B64 configuration (minibatch 3, validation ≈ 10), an accept-bearing iteration costs about $2 \times 3 + 10 = 16$ rollouts—exactly the 16 rollouts a four-way split allots to each slot. The split therefore lands the budget *at or below* the floor for every slot, and the empirical consequence is unambiguous: at 16/slot, *zero* of the four slots accepted any candidate and all four froze at the empty seed (Table 5). The structured harness that emerged was byte-identical to stock, which is why HARNESSEVO-full ties stock to within +0.015 of noise. By contrast, at 32/slot three of four slots accept a mutation but only one freezes non-empty text (the load-bearing control slot), and at 60/slot all four accept and freeze text. The floor is real, and uniform splitting drives the per-slot budget straight through it.

This yields the paper’s sharpest practical statement: *naive structured evolution can be worse than a targeted single-slot run*, because decomposing the harness divides the budget below the optimizer’s working point. The remedy is not more structure but better allocation.

7.3 Budget concentration recovers the gain

If the floor explains the failure, then concentrating the budget should fix it, and it does (Table 4, Figure 4). Spending the entire budget on the high-credit control slot keeps every rollout above the floor: the control-only run accepts and freezes a substantive rule, and held-out success rises to 0.761 at half the split’s budget (+0.119, **) and 0.724 at the full budget (+0.082, borderline). The half-budget run is, strikingly, the strongest configuration we observe—more budget is not the issue; *where* the budget goes is. This is the prescription we draw: under a fixed budget, identify the high-credit slot (via a cheap LOI probe) and concentrate the budget there rather than splitting it uniformly across named components. We report the @32 (significant) and @64 (borderline) replications together to be candid that the magnitude is noisy; the ordering—targeted $\gg$ split $\approx$ flat $\approx$ stock—is what is robust.

Budget allocation at fixed $B=64$: concentrating on c_4 recovers the gain on ALFWorld; WebShop stays null

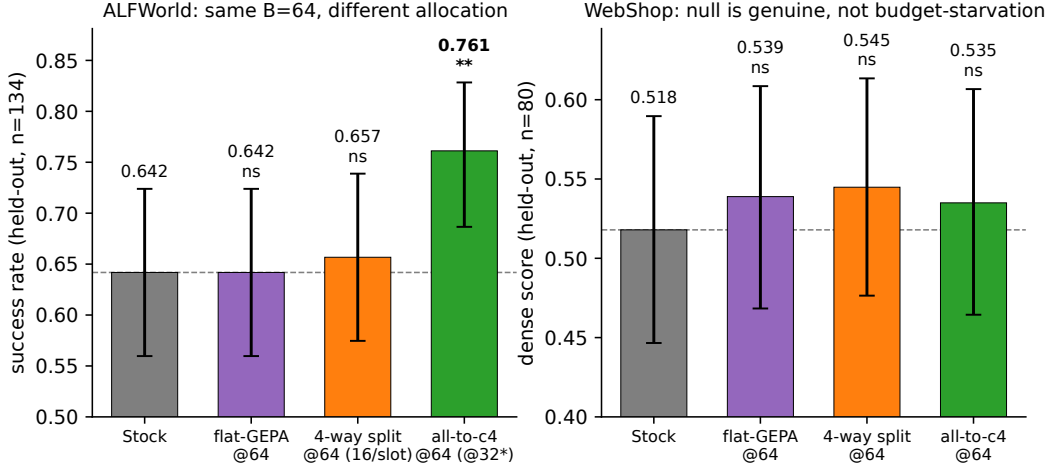


Figure 4: **Budget allocation.** Left (ALFWorld, $n=134$): at the same budget $B=64$, concentrating on the control slot (all-to- c_4 , point shown at the significant @32 replication, 0.761, **) far exceeds the four-way split (0.657, ns) and flat evolution (0.642, ns). Right (WebShop, $n=80$): the null is *genuine*, not budget starvation—even all-to- c_4 @64 stays at 0.535 (ns), because the control slot froze empty for lack of a recurrent, verbalizable control failure to fix.

7.4 Task-type breakdown: where the control rule pays off

Figure 5 breaks the control-slot gain down by ALFWorld task type (at the significant $B=32$ setting). The gains concentrate exactly where the control rule’s content predicts. The largest improvement is on `look_at_obj_in_light` tasks ($\Delta = +0.444$, $n = 18$): these require the agent not to give up and to use the desk lamp while holding the object, a classic premature-stopping / loop failure that the control rule directly addresses. The next largest is on `clean_then_place` ($\Delta = +0.226$, $n = 31$), a multi-step, closed-receptacle type where “open before taking” and “confirm holding before placing” bite. Simple `pick_and_place` ($\Delta = +0.042$, $n = 24$) and `heat_then_place` ($\Delta = +0.087$, $n = 23$) show small positive effects; the two types where the rule helps least or slightly hurts—`cool_then_place` ($\Delta = -0.048$, $n = 21$) and `pick_two_obj_and_place` ($\Delta = -0.059$, $n = 17$)—are where the generic heuristic occasionally over-fires. Of the 22 net task wins from the control rule, half (11) fall in the closed-receptacle / multi-step types it was written for. The gain is mechanistic, not diffuse.

7.5 The WebShop null is genuine task-contingency, not starvation

On WebShop every slot froze empty under HARNESSEVO-full, and the full method tied the baselines (Table 2). A natural worry is that this null is just another instance of the budget-splitting trap—perhaps WebShop too has a load-bearing slot, starved by the four-way split. We rule this out directly. The all-to-control run on WebShop, which hands the *entire* budget to the control slot (well above the accept floor), *also* returns a null: dense score 0.535 versus 0.518 for stock ($\Delta = +0.017$, paired-bootstrap $p = 0.36$, ns), and the control slot froze empty even with the concentrated budget. The optimizer had every opportunity to accept a control rule and found none worth accepting. The WebShop result is therefore a genuine property of the task, not an artifact of allocation: WebShop’s instruction-conditioned purchasing, with its dense attribute-match reward, simply does not exhibit a recurrent, verbalizable control failure mode that a general heuristic can repair. This is the boundary of the phenomenon—harness optimization

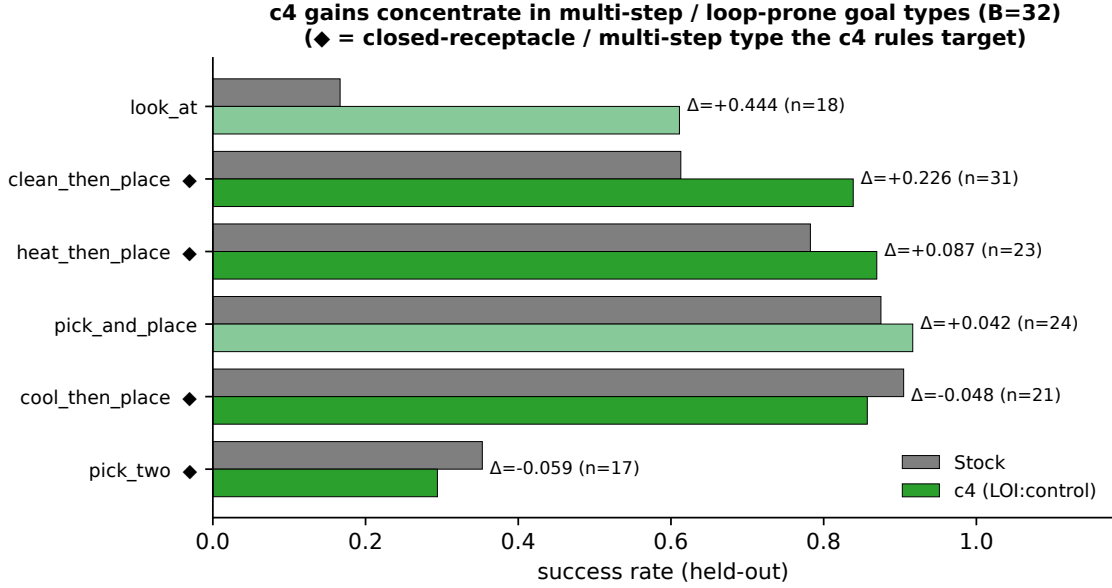


Figure 5: **Control-slot gain by ALFWorld task type** (control-only LOI at $B=32$). Diamonds (♦) mark closed-receptacle / multi-step types the control rules target. Gains concentrate in loop-prone, multi-step types (*look_at*, *clean_then_place*); near-saturated or short-horizon types show little change. The effect tracks the rule’s content.

captures value precisely when the task has recurrent, verbalizable control failures (dead-end actions, ordering constraints, premature stopping), and is null otherwise.

7.6 Grounded self-correction, not reward-hacking

The value that survives is, on inspection, environment-grounded *self-correction* rather than reward-exploiting strategy. The control rule fixes real control errors and generalizes across held-out task types (Figure 5); it does not encode a shortcut that games the binary success checker. Two observations reinforce this. First, the gain transfers across task types it was not specifically tuned on—the mark of a genuine correction rather than an over-fit. Second, on the benchmark with the more gameable dense reward (WebShop), the optimizer declines to fill any slot rather than fabricate a reward-chasing strategy. A harness optimizer that captures grounded correction where it exists and abstains where it does not is the desired behaviour in the presence of an imperfect environment reward; we develop this as a constructive counterpoint to over-optimization concerns in Section 2.4. Validation-to-test gaps were small or negative throughout, indicating no over-fitting to the selection set.

8 Discussion

Localize before you optimize. The headline practical lesson is that harness-optimization value is not diffuse across the scaffold; on our tasks it is concentrated in a single component. This changes how a fixed optimization budget should be spent. Rather than evolve the harness as one string (which cannot tell you where the value is) or split a budget uniformly across named components (which, as we show, can drive every component below the optimizer’s accept-floor and capture nothing), the budget should first be used cheaply to *locate* the high-credit component—a single-slot LOI probe suffices—and then

concentrated there. Structured decomposition is valuable as a *microscope* for this localization, not as a drop-in method for a better agent.

The budget-splitting trap is a property of the optimizer, not the harness. The sub-additivity we observe is mechanistic: it follows from the accept-and-rescore floor of the reflective optimizer. Any optimizer with a minimum budget to accept and validate a candidate will exhibit some version of this trap when its budget is divided across components below that minimum. The practical implication generalizes beyond our specific optimizer: when distributing a fixed search budget across parts of a system, account for the per-part minimum the search procedure needs to make progress, or the division will silently waste the budget.

When does harness optimization help? Our two benchmarks bracket the answer. Harness optimization helped on ALFWorld and was null on WebShop, and the audited slot text explains the difference: ALFWorld has recurrent, verbalizable control failures that a compact general rule can fix, whereas WebShop does not. This is a usable predictor. Before investing in harness optimization for a new task, ask whether the frozen agent’s failures are (i) recurrent across instances and (ii) describable as a general control heuristic. If yes, a targeted control-slot evolution is likely to pay off; if no, the effort will return a null, and—reassuringly—a well-behaved optimizer will return that null rather than an over-fit.

The surviving value is the safe kind. It is worth emphasizing that the optimization value we localize is grounded self-correction, which is exactly the kind of value one would want a black-box harness optimizer to capture: it fixes real control errors, generalizes, and does not exploit the reward. On the benchmark where the reward is most gameable, the method abstained. This is a small but encouraging signal that careful, localized harness optimization need not amount to reward-hacking.

9 Conclusion

We asked where, inside an LLM agent’s harness, optimization value lives, and we built a structured microscope to find out: HARNESSEVO, a four-slot decomposition evolved by coordinate ascent under an iso-budget, paired with a leave-one-in / leave-one-out credit-assignment protocol. The honest binary result is a tie: the full structured method matches both the stock harness and flat single-string evolution on held-out success (0.657 / 0.642 / 0.642, ns). The finding is *where* the value lives and *how budget destroys it*. On ALFWorld the value is localized almost entirely to the reflection/control slot (leave-one-in +0.119, $p = 0.0046$; role, strategy, format individually null). The structured method ties not because the value is absent but because it is strongly sub-additive: splitting an iso-budget of 64 rollouts across four slots leaves 16 per slot, below the optimizer’s accept-and-rescore floor, so every slot froze empty and the load-bearing gain was never captured—the budget-splitting trap. Concentrating the same budget on the high-credit slot recovers the discarded gain (targeted control-only: 0.761, +0.119, $p = 0.0046$ at half the budget; 0.724, +0.082, borderline at the full budget—reported together, with no overclaim at the larger budget). The phenomenon is task-contingent: on WebShop every slot froze empty, a targeted control run stayed null, and the absence is a genuine property of the task rather than starvation. The value that does survive is environment-grounded self-correction, not reward-exploitation. We conclude that credit assignment and budget concentration should precede any structured agent-evolution scheme: *localize first, then concentrate the budget on the one slot that matters*.

Limitations. We name them plainly. (1) *Single frozen backbone*. All results use one frozen Qwen2.5-7B model; we do not run a larger backbone. Whether the control-slot localization persists, shifts to

another slot, or vanishes at larger scale is the natural next study—we explicitly flag backbone-scaling of the localization as future work and make *no* claim about it here. (2) *Two benchmarks, one of them null*. We study ALFWorld and WebShop; the WebShop null is reported as a task-contingency boundary, not a weakness, but two environments cannot map the full space of when harness optimization helps. (3) *One structured scheme*. We evolve slots by coordinate ascent with a single reflective optimizer; other decompositions (different slot taxonomies, joint rather than sequential evolution, a different base optimizer) might split value differently, and our sub-additivity mechanism is tied to this optimizer’s accept-floor. (4) *Small evolve/validation splits and sample sizes*. The held-out sets are modest ($n=134$ ALFWorld, $n=80$ WebShop) and the validation sets used for candidate selection are small; this is why the budget-allocation magnitude is significant at @32 but only borderline at @64, and why we make directional rather than precise-magnitude claims there.

Future work. Beyond backbone scaling, the most promising directions are a cheap automatic *slot-credit estimator* that allocates budget proportional to estimated per-slot value (turning our manual prescription into a one-pass allocator), and a taxonomy of tasks by their control-failure profile to predict, a priori, whether targeted harness optimization will pay off.

References

- [1] Yongchao Zhou, Andrei Ioan Muresanu, Ziwen Han, Keiran Paster, Silviu Pitis, Harris Chan, and Jimmy Ba. Large language models are human-level prompt engineers. In *International Conference on Learning Representations (ICLR)*, 2023. arXiv:2211.01910.
- [2] Chengrun Yang, Xuezhi Wang, Yifeng Lu, Hanxiao Liu, Quoc V. Le, Denny Zhou, and Xinyun Chen. Large language models as optimizers. In *International Conference on Learning Representations (ICLR)*, 2024.
- [3] Chrisantha Fernando, Dylan Banarse, Henryk Michalewski, Simon Osindero, and Tim Rocktäschel. Promptbreeder: Self-referential self-improvement via prompt evolution. *arXiv preprint*, 2023. arXiv:2309.16797.
- [4] Omar Khattab, Arnav Singhvi, Paridhi Maheshwari, Zhiyuan Zhang, et al. DSPy: Compiling declarative language model calls into self-improving pipelines. In *International Conference on Learning Representations (ICLR)*, 2024.
- [5] Lakshya A. Agrawal, Shangyin Tan, Dilara Soylu, Noah Ziemis, Rishi Khare, Krista Opsahl-Ong, Arnav Singhvi, Herumb Shandilya, Michael J. Ryan, Meng Jiang, et al. GEPA: Reflective prompt evolution can outperform reinforcement learning. *arXiv preprint arXiv:2507.19457*, 2025.
- [6] Zora Zhiruo Wang, Jiayuan Mao, Daniel Fried, and Graham Neubig. Agent workflow memory. *arXiv preprint arXiv:2409.07429*, 2024.
- [7] Qizheng Zhang, Changran Hu, et al. Agentic context engineering: Evolving contexts for self-improving language models. *arXiv preprint arXiv:2510.04618*, 2025.
- [8] Mohit Shridhar, Xingdi Yuan, Marc-Alexandre Côté, Yonatan Bisk, Adam Trischler, and Matthew Hausknecht. ALFWorld: Aligning text and embodied environments for interactive learning. In *International Conference on Learning Representations (ICLR)*, 2021. arXiv:2010.03768.

- [9] Shunyu Yao, Howard Chen, John Yang, and Karthik Narasimhan. WebShop: Towards scalable real-world web interaction with grounded language agents. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022. arXiv:2207.01206.
- [10] Omar Khattab, Keshav Santhanam, Xiang Lisa Li, David Hall, Percy Liang, Christopher Potts, and Matei Zaharia. Demonstrate-search-predict: Composing retrieval and language models for knowledge-intensive NLP. *arXiv preprint*, 2022. arXiv:2212.14024.
- [11] Mirac Suzgun, Mert Yuksekgonul, Federico Bianchi, Dan Jurafsky, and James Zou. Dynamic cheatsheet: Test-time learning with adaptive memory. *arXiv preprint arXiv:2504.07952*, 2025.
- [12] Charlie Snell, Jaehoon Lee, Kelvin Xu, and Aviral Kumar. Scaling LLM test-time compute optimally can be more effective than scaling model parameters. *arXiv preprint*, 2024. arXiv:2408.03314.
- [13] Niklas Muennighoff, Zitong Yang, Weijia Shi, Xiang Lisa Li, Li Fei-Fei, Hannaneh Hajishirzi, Luke Zettlemoyer, Percy Liang, Emmanuel Candès, and Tatsunori Hashimoto. sl: Simple test-time scaling. *arXiv preprint*, 2025. arXiv:2501.19393.
- [14] Charles Packer, Sarah Wooders, Kevin Lin, Vivian Fang, Shishir G. Patil, Ion Stoica, and Joseph E. Gonzalez. MemGPT: Towards LLMs as operating systems. *arXiv preprint arXiv:2310.08560*, 2023.
- [15] Wanjun Zhong, Lianghong Guo, Qiqi Gao, He Ye, and Yanlin Wang. MemoryBank: Enhancing large language models with long-term memory. In *Proceedings of the AAAI Conference on Artificial Intelligence (AAAI)*, 2024. arXiv:2305.10250.
- [16] Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. Voyager: An open-ended embodied agent with large language models. *arXiv preprint arXiv:2305.16291*, 2023.
- [17] Andrew Zhao, Daniel Huang, Quentin Xu, Matthieu Lin, Yong-Jin Liu, and Gao Huang. ExpeL: LLM agents are experiential learners. In *AAAI Conference on Artificial Intelligence*, 2024.
- [18] Jiaxin Huang, Shixiang Shane Gu, Le Hou, Yuexin Wu, Xuezhi Wang, Hongkun Yu, and Jiawei Han. Large language models can self-improve. *arXiv preprint*, 2022. arXiv:2210.11610.
- [19] Eric Zelikman, Yuhuai Wu, Jesse Mu, and Noah D. Goodman. STaR: Bootstrapping reasoning with reasoning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.
- [20] Weizhe Yuan, Richard Yuanzhe Pang, Kyunghyun Cho, Sainbayar Sukhbaatar, Jing Xu, and Jason Weston. Self-rewarding language models. *arXiv preprint*, 2024. arXiv:2401.10020.
- [21] Tianhao Wu, Weizhe Yuan, Olga Golovneva, Jing Xu, Yuandong Tian, Jiantao Jiao, Jason Weston, and Sainbayar Sukhbaatar. Meta-rewarding language models: Self-improving alignment with LLM-as-a-meta-judge. *arXiv preprint*, 2024. arXiv:2407.19594.
- [22] Joon Sung Park, Joseph C. O’Brien, Carrie J. Cai, Meredith Ringel Morris, Percy Liang, and Michael S. Bernstein. Generative agents: Interactive simulacra of human behavior. In *ACM Symposium on User Interface Software and Technology (UIST)*, 2023.
- [23] Shuo Yang, Soyeon Caren Han, Xueqi Ma, Yan Li, Mohammad Reza Ghasemi Madani, and Eduard Hovy. EvoTool: Self-evolving tool-use policy optimization in LLM agents via blame-aware mutation and diversity-aware selection, 2026. ACL 2026.

- [24] Hang Su, Jun Luo, Chang Liu, Xiao Yang, Yichi Zhang, Yinpeng Dong, and Jun Zhu. A survey on autonomy-induced security risks in large model-based agents. *arXiv preprint arXiv:2506.23844*, 2025.
- [25] Jiangjie Chen, Xintao Wang, Rui Xu, Siyu Yuan, Yikai Zhang, Wei Shi, Jian Xie, Shuang Li, Ruihan Yang, Tinghui Zhu, Aili Chen, Nianqi Li, Lida Chen, Caiyu Hu, Siye Wu, Scott Ren, Ziquan Fu, and Yanghua Xiao. From persona to personalization: A survey on role-playing language agents. *Transactions on Machine Learning Research (TMLR)*, 2024. arXiv:2404.18231.
- [26] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations (ICLR)*, 2023.
- [27] Noah Shinn, Federico Cassano, Edward Berman, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.
- [28] Shunyu Yao, Dian Yu, Jeffrey Zhao, Izhak Shafran, Thomas L. Griffiths, Yuan Cao, and Karthik Narasimhan. Tree of thoughts: Deliberate problem solving with large language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023. arXiv:2305.10601.
- [29] Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc Le, Ed Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. Self-consistency improves chain of thought reasoning in language models. In *International Conference on Learning Representations (ICLR)*, 2023. arXiv:2203.11171.
- [30] Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, et al. Self-refine: Iterative refinement with self-feedback. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.
- [31] Zhibin Gou, Zhihong Shao, Yeyun Gong, Yelong Shen, Yujiu Yang, Nan Duan, and Weizhu Chen. CRITIC: Large language models can self-correct with tool-interactive critiquing. *arXiv preprint*, 2023. arXiv:2305.11738.
- [32] Jie Huang, Xinyun Chen, Swaroop Mishra, Huaixiu Steven Zheng, Adams Wei Yu, Xinying Song, and Denny Zhou. Large language models cannot self-correct reasoning yet. *International Conference on Learning Representations (ICLR)*, 2024. arXiv:2310.01798.
- [33] Ryo Kamoi, Yusen Zhang, Nan Zhang, Jiawei Han, and Rui Zhang. When can LLMs actually correct their own mistakes? a critical survey of self-correction of LLMs. *Transactions of the Association for Computational Linguistics (TACL)*, 12:1417–1440, 2024.
- [34] Yujia Qin, Shihao Liang, Yining Ye, Kunlun Zhu, Lan Yan, Yaxi Lu, Yankai Lin, Xin Cong, Xiangru Tang, Bill Qian, Sihan Zhao, Lauren Hong, Runchu Tian, Ruobing Xie, Jie Zhou, Mark Gerstein, Dahai Li, Zhiyuan Liu, and Maosong Sun. ToolLLM: Facilitating large language models to master 16000+ real-world APIs. In *International Conference on Learning Representations (ICLR)*, 2024. Spotlight; arXiv:2307.16789.
- [35] Shuo Yang, Soyeon Caren Han, Yihao Ding, Shuhe Wang, and Eduard Hovy. ToolTree: Efficient LLM agent tool planning via dual-feedback monte carlo tree search and bidirectional pruning, 2026. ICLR 2026.

- [36] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2020. arXiv:2005.11401.
- [37] Kelvin Guu, Kenton Lee, Zora Tung, Panupong Pasupat, and Ming-Wei Chang. REALM: Retrieval-augmented language model pre-training. In *International Conference on Machine Learning (ICML)*, 2020. arXiv:2002.08909.
- [38] Akari Asai, Zeqiu Wu, Yizhong Wang, Avirup Sil, and Hannaneh Hajishirzi. Self-RAG: Learning to retrieve, generate, and critique through self-reflection. In *International Conference on Learning Representations (ICLR)*, 2024. arXiv:2310.11511.
- [39] Shuyan Zhou, Frank F. Xu, Hao Zhu, Xuhui Zhou, Robert Lo, Abishek Sridhar, Xianyi Cheng, Tianyue Ou, Yonatan Bisk, Daniel Fried, Uri Alon, and Graham Neubig. WebArena: A realistic web environment for building autonomous agents. In *International Conference on Learning Representations (ICLR)*, 2024. arXiv:2307.13854.
- [40] Grégoire Mialon, Clémentine Fourrier, Craig Swift, Thomas Wolf, Yann LeCun, and Thomas Scialom. GAIA: A benchmark for general AI assistants. In *International Conference on Learning Representations (ICLR)*, 2024. arXiv:2311.12983.
- [41] Xiao Liu, Hao Yu, Hanchen Zhang, Yifan Xu, Xuanyu Lei, Hanyu Lai, Yu Gu, Hangliang Ding, Kaiwen Men, Kejuan Yang, Shudan Zhang, Xiang Deng, Aohan Zeng, Zhengxiao Du, Chenhui Zhang, Sheng Shen, Tianjun Zhang, Yu Su, Huan Sun, Minlie Huang, Yuxiao Dong, and Jie Tang. AgentBench: Evaluating LLMs as agents. In *International Conference on Learning Representations (ICLR)*, 2024. arXiv:2308.03688.
- [42] Shunyu Yao, Noah Shinn, Pedram Razavi, and Karthik Narasimhan. τ -bench: A benchmark for tool-agent-user interaction in real-world domains. *arXiv preprint arXiv:2406.12045*, 2024.
- [43] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? In *International Conference on Learning Representations (ICLR)*, 2024. arXiv:2310.06770.
- [44] Ari S. Morcos, David G.T. Barrett, Neil C. Rabinowitz, and Matthew Botvinick. On the importance of single directions for generalization. In *International Conference on Learning Representations (ICLR)*, 2018. arXiv:1803.06959.
- [45] Swarnadeep Saha, Omer Levy, Asli Celikyilmaz, Mohit Bansal, Jason Weston, and Xian Li. Branch-solve-merge improves large language model evaluation and generation. *arXiv preprint arXiv:2310.15123*, 2023.
- [46] Dawei Li, Bohan Jiang, Liangjie Huang, Alimohammad Beigi, Chengshuai Zhao, Zhen Tan, Amrita Bhattacharjee, Yuxuan Jiang, Canyu Chen, Tianhao Wu, Kai Shu, Lu Cheng, and Huan Liu. From generation to judgment: Opportunities and challenges of LLM-as-a-judge. *arXiv preprint arXiv:2411.16594*, 2024.
- [47] Johannes von Oswald, Eyvind Niklasson, Ettore Randazzo, João Sacramento, Alexander Mordvintsev, Andrey Zhmoginov, and Max Vladymyrov. Transformers learn in-context by gradient descent. In *International Conference on Machine Learning (ICML)*, 2023. arXiv:2212.07677.

- [48] Ekin Akyürek, Dale Schuurmans, Jacob Andreas, Tengyu Ma, and Denny Zhou. What learning algorithm is in-context learning? investigations with linear models. In *International Conference on Learning Representations (ICLR)*, 2023. arXiv:2211.15661.
- [49] Charles A. E. Goodhart. Problems of monetary management: The UK experience. *Monetary Theory and Practice: The UK Experience*, pages 91–121, 1984.
- [50] Leo Gao, John Schulman, and Jacob Hilton. Scaling laws for reward model overoptimization. In *International Conference on Machine Learning (ICML)*, 2023. arXiv:2210.10760.
- [51] Joar Skalse, Nikolaus H. R. Howe, Dmitrii Krashennnikov, and David Krueger. Defining and characterizing reward hacking. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022. arXiv:2209.13085.
- [52] Alexander Pan, Kush Bhatia, and Jacob Steinhardt. The effects of reward misspecification: Mapping and mitigating misaligned models. In *International Conference on Learning Representations (ICLR)*, 2022. arXiv:2201.03544.
- [53] Dario Amodei, Chris Olah, Jacob Steinhardt, Paul Christiano, John Schulman, and Dan Mané. Concrete problems in AI safety. *arXiv preprint arXiv:1606.06565*, 2016.
- [54] Nisan Stiennon, Long Ouyang, Jeff Wu, Daniel M. Ziegler, Ryan Lowe, Chelsea Voss, Alec Radford, Dario Amodei, and Paul Christiano. Learning to summarize from human feedback. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2020. arXiv:2009.01325.
- [55] Rafael Rafailov, Archit Sharma, Eric Mitchell, Stefano Ermon, Christopher D. Manning, and Chelsea Finn. Direct preference optimization: Your language model is secretly a reward model. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023. arXiv:2305.18290.
- [56] Harrison Lee, Samrat Phatale, Hassan Mansoor, Thomas Mesnard, Johan Ferret, Kellie Lu, Colton Bishop, Ethan Hall, Victor Carbune, Abhinav Rastogi, and Sushant Prakash. RLAI vs. RLHF: Scaling reinforcement learning from human feedback with AI feedback. In *Proceedings of the 41st International Conference on Machine Learning (ICML)*, 2024. arXiv:2309.00267.
- [57] Shuo Yang, Soyeon Caren Han, Shuhe Wang, Yan Li, Yihao Ding, and Eduard Hovy. Toward understanding misalignment in LLM agents: A survey of taxonomy, causes, mitigation, and evaluation, 2026. URL <https://openreview.net/forum?id=zzTEGP2BYa>. ACL ARR 2026.
- [58] Yuntao Bai, Saurav Kadavath, Sandipan Kundu, Amanda Askell, Jackson Kernion, Andy Jones, Anna Chen, Anna Goldie, Azalia Mirhoseini, Cameron McKinnon, et al. Constitutional AI: Harmlessness from AI feedback. *arXiv preprint*, 2022. arXiv:2212.08073.
- [59] Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll L. Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, John Schulman, Jacob Hilton, Fraser Kelton, Luke Miller, Maddie Simens, Amanda Askell, Peter Welinder, Paul Christiano, Jan Leike, and Ryan Lowe. Training language models to follow instructions with human feedback. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022. arXiv:2203.02155.
- [60] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhonghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric P. Xing, Hao Zhang, Joseph E. Gonzalez, and Ion Stoica. Judging LLM-as-a-judge with MT-Bench and Chatbot Arena. In *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*, volume 36, 2023. arXiv:2306.05685.

- [61] Yang Liu, Dan Iter, Yichong Xu, Shuohang Wang, Ruochen Xu, and Chenguang Zhu. G-Eval: NLG evaluation using GPT-4 with better human alignment. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 2511–2522, 2023. doi: 10.18653/v1/2023.emnlp-main.153. arXiv:2303.16634.
- [62] Seungone Kim, Jamin Shin, Yejin Cho, Joel Jang, Shayne Longpre, Hwaran Lee, Sangdoo Yun, Seongjin Shin, Sungdong Kim, James Thorne, and Minjoon Seo. Prometheus: Inducing fine-grained evaluation capability in language models. In *International Conference on Learning Representations (ICLR)*, 2024. arXiv:2310.08491.
- [63] Jiawei Gu, Xuhui Jiang, Zhichao Shi, Hexiang Tan, Xuehao Zhai, Chengjin Xu, Wei Li, Yinghan Shen, Shengjie Ma, Honghao Liu, Saizhuo Wang, Kun Zhang, Yuanzhuo Wang, Wen Gao, Lionel Ni, and Jian Guo. A survey on LLM-as-a-judge. *arXiv preprint arXiv:2411.15594*, 2024.
- [64] Arjun Panickssery, Samuel R. Bowman, and Shi Feng. LLM evaluators recognize and favor their own generations. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 37, 2024. arXiv:2404.13076.
- [65] Xiyuan Yang, Jiaru Zou, Rui Pan, Ruizhong Qiu, Pan Lu, Shizhe Diao, Jindong Jiang, Hanghang Tong, Tong Zhang, Markus J. Buehler, Jingrui He, and James Zou. Recursive multi-agent systems. *arXiv preprint arXiv:2604.25917*, 2026.
- [66] Jiaru Zou, Ruizhong Qiu, Gaotang Li, Xiyuan Yang, Katherine Tieu, Pan Lu, Ke Shen, Hanghang Tong, Yejin Choi, Jingrui He, James Zou, Mengdi Wang, and Ling Yang. Latent collaboration in multi-agent systems. *arXiv preprint arXiv:2511.20639*, 2025. ICML 2026 Spotlight.
- [67] An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, et al. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025.

A Frozen Slot Texts, Additional Arms, and Hyperparameters

A.1 Audited frozen slot texts

The localization claim rests on auditable text. We reproduce the control-slot content that produced the significant ALFWorld gain (the $B=32$ control-only run, 401 characters, frozen after validation selection):

```
think: Before performing any action, ensure the previous step succeeded.
if action returned "Nothing happens" and is identical to the last action, skip it
and try another location.
open any closed receptacle before taking from it.
confirm the object is held before attempting to place it.
stop the sequence once the task is completed successfully or all relevant locations
are checked without success.
```

The full-budget all-to-control run ($B=64$) froze a longer, pseudo-code variant of the same rules (1000 characters; an *if/then* elaboration of “skip repeated dead-end actions, open before taking, verify holding before placing, stop when satisfied”). On **WebShop**, by contrast, the control slot (and every other slot, under both the four-way split and the all-to-control run) froze *empty* (0 characters): the optimizer accepted no candidate. This empty-versus-substantive contrast across benchmarks is the direct, auditable signature of the task-contingency boundary in Section 7. The role, strategy, and format slots on ALFWorld also froze empty in the iso-budget full run (0 characters each), consistent with their null LOI effects.

Table 6: B120 corroboration arm on ALFWorld ($n=134$, 10k-bootstrap CI). Main methods (top) and per-component LOI map (bottom). All differences ns; the control direction remains the largest single-slot gain and the system is sub-additive.

Method / Slot	Success / Δ^{LOI} [95% CI]	McNemar p	Rollouts
Stock	0.672 [0.590, 0.746]	—	0
GEPA-flat	0.642 [0.560, 0.716]	0.343 ns	120
HARNESSEVO-full	0.627 [0.545, 0.709]	0.377 ns	120
c_1 role	+0.007 [−0.02, +0.04]	1.000 ns	60
c_2 strategy	+0.015 [−0.07, +0.10]	0.860 ns	60
c_3 format	−0.022 [−0.07, +0.02]	0.546 ns	60
c_4 control	+0.052 [−0.02, +0.13]	0.265 ns	60

A.2 B120 corroboration arm (full table)

Table 6 gives the higher-budget arm in full. Stock measures higher at this configuration (0.672), flat evolution gives 0.642, and HARNESSEVO-full gives 0.627; all pairwise McNemar tests are ns. The control slot remains the most positive single-slot direction ($\Delta_{c_4}^{\text{LOI}} = +0.052$, ns), and the system is sub-additive (full -0.045 vs $\sum \text{LOI} = +0.052$). We include this arm as a robustness check on the *ordering* of per-slot value, not as an independent significant result.

A.3 Hyperparameters and protocol

- **Backbone:** frozen Qwen2.5-7B, used as both task policy and reflection model; rollouts deterministic (temperature=0); reflection mutations sampled (temperature 0.9), as in GEPA.
- **ALFWorld:** train $|T|=30$ (balanced over task types from a 30-task dev manifest), validation ≈ 10 (B64) / 8 (B120), test $n=134$ (disjoint held-out manifest, zero leakage); max 40 steps/episode; binary environment-verified success.
- **WebShop:** disjoint train/validation/test sessions, test $n=80$; dense $[0, 1]$ attribute-match score.
- **Optimizer (GEPA):** instance-Pareto parent selection over a growing candidate pool; reflective mutation on a train minibatch (size 3 at B64, 2 at B120); improvement-or-equal accept gate for the binary-sparse reward; accepted children re-scored on the full validation set; final = best-validation candidate, frozen. Iso-budget: per-slot budgets sum exactly to the flat method’s budget.
- **Budgets:** $B=64$ (primary), $B=120$ (corroboration); single-slot LOI at $B/2$; all-to-control at 32 and 64.
- **Statistics:** 10,000-resample bootstrap 95% CIs; paired McNemar (ALFWorld binary) / paired bootstrap (WebShop dense); ns labeled; */ **/ *** for $p < 0.05/0.01/0.001$.
- **Accept-floor diagnostic:** an accept-bearing GEPA iteration costs $2 \times \text{minibatch} + |V|$ rollouts; at B64 (minibatch=3, $|V|\approx 10$) this is ≈ 16 , equal to the four-way split’s 16/slot, which is why the split froze every slot empty (Table 5).